SentinelX — Technical Artifact: Key Code Explanation

1. Executive Summary of Architectural Integrity

SentinelX enforces a strict **human-gated triage architecture** that decouples the AI inference engine from autonomous decision-making loops, satisfying Track A compliance guidelines. The backend never acts on AI output without an explicit analyst trigger via the dashboard UI. Data remains inert in MongoDB as a static, unacted-upon record until an authenticated agency user clicks “Analyze.” The WebSocket layer broadcasts events exclusively to the analyst’s room, ensuring no automated dispatch path exists outside the manual review loop.

---

2. Core Backend Production Script (Human-Gated Triage Route)

File: `besafe_app.py` — `POST /agency/reports/<report_id>/analyze`

```
@app.route("/agency/reports/<report_id>/analyze", methods=["POST"])
@jwt_required()
def agency_analyze_report(report_id):
    agency_id = get_jwt_identity()

    #      HUMAN GATE 1: Read-only fetch from MongoDB
    # The report sits inert in the database. No AI processing
    # happens until an authenticated analyst requests it.
    report = get_report_by_id(report_id)
    if not report:
        return jsonify({"error": "Report not found"}), 404
    if str(report.get("assignedAgencyId")) != agency_id:
        return jsonify({"error": "Report not found"}), 404

    #      Structured intake fields extracted
```

```
# These four dimensions (category, narrative, timing, frequency)
# form the feature vector passed to the LLM.
category    = report.get("category", "")
description = report.get("description", "")
timing      = report.get("timing", "")
frequency   = report.get("frequency", "")
attachments = report.get("attachments", [])

#      HUMAN GATE 2: Analyst-triggered dispatch
# Only now, on explicit click, does the system call the AI.
# The dispatcher routes to the configured provider:
#   "openai"      GPT-4o with structured output (Pydantic schema)
#   "gemini"      Gemini 2.0 Flash with JSON response mode
#   "freemodel"    gpt-4o-mini via FreeModel with json_object mode
analysis = call_gpt_model(
    category, description, timing, frequency,
    attachments=attachments
)

#      Pipeline error guard
# If the LLM returns an error shape, surface it immediately
# rather than writing a malformed document.
if analysis.get("identified_pattern_type") == "Pipeline_Error":
    detail = analysis.get("error_message", "Unknown AI error")
    return jsonify({"error": "AI analysis failed", "detail": detail}), 500

#      Persist analysis to MongoDB
# The result is written back to the same document so the
# dashboard can re-render the detail panel on next load.
updated = update_report_analysis(report_id, analysis)
if not updated:
    return jsonify({"error": "Failed to update report"}), 500

#      WebSocket push to the analyst's room
# 'report_analyzed' is emitted so the live dashboard shows the
# AI analysis card without a page refresh. No autonomous
# action is taken — the analyst must still review and decide.
socketio.emit("report_analyzed", {
    "report_id": report_id,
    "ai_Analysis": analysis,
```

```
    }, room=f"agency_{agency_id}")

    return jsonify({"success": True, "analysis": analysis})
```

Supporting model pipeline (Bi-LSTM threat classifier)

The custom **Bi-LSTM model** is loaded via ONNX Runtime in `model_pipeline.py`. It is used in the **SOS audio alert pipeline** (not the report analysis endpoint above) to calculate linguistic threat probability from transcribed speech:

```
#      model_pipeline.py — ONNX Bi-LSTM inference

import onnxruntime as ort
from keras_preprocessing.text import tokenizer_from_json

# Load the tokenizer fitted during training
with open("tokenizers/BesafeV1_1.0.00.json") as f:
    tokenizer = tokenizer_from_json(f.read())

# Load the exported ONNX graph (Bi-LSTM    Dense    Sigmoid)
session = ort.InferenceSession("model/BesafeV1_1.0.00.onnx")

def predict_threat(text, threshold=0.5):
    # Tokenize    pad    run ONNX session
    processed = preprocess(text)
    confidence = session.run(None, {input_name: processed})[0][0][0]
    confidence = round(float(confidence), 2)
    label = "Threat" if confidence > threshold else "Non-Threat"
    return label, confidence, MODEL_VERSION
```

Priority scoring compound formula

The dashboard's urgency sort uses a weighted compound formula that blends the Bi-LSTM confidence score, temporal decay, and acknowledgment state:

```
#      utils.py — Dynamic priority scoring

def calculate_priority(alert: dict) -> float:
    """
```

```
    Formula:
      priority = (confidence × 0.6)      Bi-LSTM threat probability
               + (time_decay  × 0.3)      minutes since creation (caps at 30 min)
               + (unacked     × 0.1)      1.0 if status is 'active'

    Result range: 0.0    1.0  (higher = more urgent)
    """
    confidence = float(alert.get("confidence", 0))
    created_at = parse_timestamp(alert.get("created_at"))
    minutes_old = (datetime.now() - created_at).total_seconds() / 60
    time_weight = min(minutes_old / 30.0, 1.0)
    unacked = 1.0 if alert.get("status") == "active" else 0.0

    priority = (confidence * 0.6) + (time_weight * 0.3) + (unacked * 0.1)
    return round(priority, 4)
```

≥ 0.75 confidence guardrail (alert filter)

The `priority_label()` function maps the compound score to a severity band. The **≥ 0.75 threshold** acts as the alert filter that promotes an item to `CRITICAL` in the dashboard badge count and the overview statistics:

```
def priority_label(score: float) -> str:
    if score >= 0.75:       #    CRITICAL guardrail — triggers alert badge
        return "CRITICAL"
    elif score >= 0.50:
        return "HIGH"
    elif score >= 0.25:
        return "MEDIUM"
    return "LOW"
```

WebSocket emission for live triage

When an alert is created (via SOS or threat detection), the socket event `report_analyzed` (for reports) or `new_alert` (for SOS) is emitted exclusively to the agency's dedicated room:

```
#      Socket.IO room-scoped emission
socketio.emit("report_analyzed", {
```

```
    "report_id": report_id,
    "ai_Analysis": analysis,
}, room=f"agency_{agency_id}")
```

No broadcast — only the assigned agency dashboard receives the event, preserving data isolation.

---

3. Core Frontend Production Script (4-Step Intake Wizard Component)

File: mobile-app/components/safechat/GuidedChatView.tsx

```
import { useEffect, useRef, useState } from "react"
import { View, Text, TouchableOpacity, TextInput, ScrollView,
        StyleSheet, Alert, Modal, KeyboardAvoidingView, Platform } from "react-native"
import { Ionicons } from "@expo/vector-icons"
import * as Location from "expo-location"
import { submitReport, uploadFile } from "@/services/safechat.service"

//      Four structured intake fields
// These map directly to the backend's analysis feature vector.
// category is preset from the scenario selector on the previous screen.

type Category = "abuse-home" | "harassment" | "unsafe-ride" | "threats" | "other"
type Step = "description" | "evidence" | "timing" | "frequency" | "outcome" | "complete

// Timing and frequency are single-select picklists (no free text)
// to eliminate unrestricted media spam and protect victim privacy.
const TIMING_OPTIONS = [
  { key: "just-now",    label: "Just now" },
  { key: "today",       label: "Today" },
  { key: "this-week",   label: "This week" },
  { key: "longer-ago",  label: "Longer ago" },
]
const FREQUENCY_OPTIONS = [
  { key: "first", label: "First time" },
  { key: "few",   label: "A few times" },
  { key: "many",  label: "Many times" },
```

```
]

export default function GuidedChatView({ category, onClose, onEscalate }: Props) {
  const insets = useSafeAreaInsets()
  const [step, setStep] = useState<Step>("description")
  const [description, setDescription] = useState("")   // free-text narrative
  const [timing, setTiming] = useState<string | null>(null)        // structured field
  const [frequency, setFrequency] = useState<string | null>(null)   // structured field
  const [attachments, setAttachments] = useState<Attachment[]>([])  // media evidence
  const [submitting, setSubmitting] = useState(false)

  //      Step 1: Description
  const renderDescriptionStep = () => (
    <KeyboardAvoidingView>
      <Text style={styles.stepTitle}>Tell us what happened</Text>
      <TextInput
        style={styles.textArea}
        placeholder="Describe what happened..."
        multiline
        value={description}
        onChangeText={setDescription}
        autoFocus
      />
      <TouchableOpacity disabled={description.trim().length < 3} onPress={next}>
        <Text>Next</Text>
      </TouchableOpacity>
    </KeyboardAvoidingView>
  )

  //      Step 2: Evidence (optional, bounded)
  // Users may attach photos, audio, or documents. Backend enforces
  // a 10 MB per-file limit and a whitelist of allowed extensions.
  // No unrestricted media — only known safe types pass through.
  const renderEvidenceStep = () => (
    <View>
      <Text style={styles.stepTitle}>Add evidence</Text>
      <Text style={styles.stepSubtitle}>
        Attach photos, audio, or documents. This step is optional.
      </Text>
      <EvidenceCapture attachments={attachments} onChange={setAttachments} />
```

```
  </View>
)

//      Step 3: Timing (structured picklist)
// Single-select card list. Eliminates free-text date ambiguity.
const renderTimingStep = () => (
  <View>
    <Text style={styles.stepTitle}>When did this happen?</Text>
    {TIMING_OPTIONS.map((o) => (
      <TouchableOpacity
        key={o.key}
        style={[styles.optionCard, timing === o.key && styles.optionCardSelected]}
        onPress={() => { setTiming(o.key); setStep("frequency") }}
      >
        <Text style={[styles.optionLabel, timing === o.key && styles.optionLabelSelected]}>
          {o.label}
        </Text>
        {timing === o.key && <Ionicons name="checkmark-circle" size={20} color="#353FA
      </TouchableOpacity>
    ))}
  </View>
)

//      Step 4: Frequency (structured picklist)
// Single-select card list. Encodes repeat-exposure risk
// without requiring the victim to relive details.
const renderFrequencyStep = () => (
  <View>
    <Text style={styles.stepTitle}>How often has this happened?</Text>
    {FREQUENCY_OPTIONS.map((o) => (
      <TouchableOpacity
        key={o.key}
        style={[styles.optionCard, frequency === o.key && styles.optionCardSelected]}
        onPress={() => { setFrequency(o.key); setStep("outcome") }}
      >
        <Text style={[styles.optionLabel, frequency === o.key && styles.optionLabelSelecte
          {o.label}
        </Text>
        {frequency === o.key && <Ionicons name="checkmark-circle" size={20} color="#35
      </TouchableOpacity>
```

```
    ))}
  </View>
)

//      Outcome: Privacy-preserving submission
// Three paths: Keep Private (device-only), Submit for Help
// (agency-routed), or SOS (immediate escalation).
const renderOutcomeStep = () => (
  <View>
    <TouchableOpacity style={styles.outcomeCard} onPress={handleKeepPrivate}>
      <Ionicons name="lock-closed" size={24} color="#17c983" />
      <Text>Keep Private — saved only on this device</Text>
    </TouchableOpacity>
    <TouchableOpacity style={styles.outcomeCard} onPress={() => setShowConsent(true
      <Ionicons name="shield-checkmark" size={24} color="#353FAB" />
      <Text>Submit for Help — sent to a response agency</Text>
    </TouchableOpacity>
    <TouchableOpacity style={[styles.outcomeCard, styles.sosCard]} onPress={handleSo
      <Ionicons name="warning" size={24} color="#ff2d3a" />
      <Text>Send SOS Now — alert agencies and contacts immediately</Text>
    </TouchableOpacity>
  </View>
)

//      Step indicator (4-dot progress bar)
// Visual feedback showing position in the intake sequence.
const stepsCompleted = (current: Step, dot: Step) => {
  const order: Step[] = ["description", "evidence", "timing", "frequency", "outcome"]
  return order.indexOf(current) >= order.indexOf(dot)
}

return (
  <View style={[styles.container, { paddingTop: insets.top }]}>
    <View style={styles.header}>
      <Text style={styles.headerTitle}>Safe Chat</Text>
      {step !== "complete" && (
        <View style={styles.progress}>
          {(["description", "evidence", "timing", "frequency"] as Step[]).map((s) => (
            <View key={s} style={[styles.dot, stepsCompleted(step, s) && styles.dotActive]} />
          ))}
```

```
          </View>
        )}
      </View>
      <ScrollView style={styles.body}>
        {step === "description" && renderDescriptionStep()}
        {step === "evidence"    && renderEvidenceStep()}
        {step === "timing"      && renderTimingStep()}
        {step === "frequency"   && renderFrequencyStep()}
        {step === "outcome"     && renderOutcomeStep()}
        {step === "complete"    && renderCompleteStep()}
      </ScrollView>
    </View>
  )
}
```

Privacy and spam-protection design

Mechanism	Implementation
Bounded evidence types	Backend whitelists `{jpg, jpeg, png, webp, heic, m4a, mp3, wav, aac, mp4, mov, avi, mkv, webm, pdf, doc, docx, txt}` — all other file types rejected
Per-file size limit	10 MB enforced at `safechat/routes.py:64` before any processing
Victim location privacy	Location is only captured with explicit `expo-location` permission prompt at submission time (line 125-128)
Private vs. submitted duality	"Keep Private" stores the report locally via `useReportStorage` (AsyncStorage); never transits the network
Consent gate before agency routing	A modal (`showConsent`) with explicit "Submit for Help" confirmation prevents accidental sharing

The 4-step wizard reduces the victim's cognitive load by presenting one discrete decision per screen, while the structured picklists for timing and frequency eliminate free-text ambiguity and block injection-style spam vectors.